Isolon: Keeping Isolation Claims Honest in a Bare-Metal Type-1 Hypervisor

Anonymous Author(s)
Submission Id: 394

Abstract

Isolon is a clean-slate Type-1 hypervisor that boots as one UEFI binary on a Dell PowerEdge R640, where it runs unmodified Linux 6.12 with virtio-blk, virtio-net, dual-vCPU SMP, and three stub guests on the same boot, then serves HTTP from a host-owned NIC.

Attaching a machine-checked isolation theorem to that binary is a labelling problem before it is a proof problem: the artifact that is proved and the mapping path the guests actually use need not be the same object. Isolon's proved artifact is guest-exclusivity in a host-only Verus ghost model. Its production guest-physical coverage at Linux boot is a 16-slot range registry the ghost model never sees. The two are disjoint, not approximate, and no one-axis "verified" label makes that visible.

We propose a two-axis ladder: proof maturity (L0–L3) against artifact (ghost model, executable module, binary on the target). Applied to eight systems, it names the axis the seL4 team crossed over four years from C to ARM binary, separates a SeKVM abstract that reads as a proved hypervisor from a Coq overlay on a KVM module, and would have split an earlier draft of this paper that put a ghost-model theorem beside a bare-metal boot. Six lessons generalized while shipping the binary; the sharpest is that a ghost model written against working code inherits that code's blind spots, so derive it from the threat model's domain instead.

CCS Concepts: • **Software and its engineering** → **Software verification**; **Virtual machines**; • **Security and privacy** → *Logic and verification.*

Keywords: formal verification, Type-1 hypervisor, EPT, Verus, maturity ladder, experience

ACM Reference Format:
Anonymous Author(s). 2027. Isolon: Keeping Isolation Claims Honest in a Bare-Metal Type-1 Hypervisor. In *Proceedings of European Conference on Computer Systems (EuroSys '27).* ACM, New York, NY, USA, 11 pages.

1 Introduction

A hypervisor's job is to make one computer look like several, without letting those several computers look at each other. On modern Intel servers the mechanism that actually enforces that is Extended Page Tables (EPT) [10]. If EPT is wrong, two guests share a frame, or a guest sees hypervisor memory, and no amount of management-API polish repairs it.

Commercial Type-1 stacks—ESXi, Xen [1], KVM [11]—are large, mature, and tested. They are not accompanied by a machine-checked isolation theorem for the frames they hand to guests. Research systems that *are* proved (seL4 [12, 13], CertiKOS [7], Komodo [5]) target microkernels, certified kernels, or enclaves. They are not a single UEFI binary aimed at the PowerEdge fleet already in the rack.

Isolon is a clean-slate Type-1 hypervisor written in Rust. It compiles to one `.efi` and boots on a Dell PowerEdge R640: Intel Xeon Silver 4110 at 2.10 GHz (family 6, model 0x55, stepping 4), BIOS 2.2.11, boot map `pages_above_1MiB`=16,574,029 (≈63.2 GiB), iDRAC virtual floppy, COM2. Formal verification is an architectural constraint, not a post-hoc whitepaper. We picked Verus [15, 16] for SMT proofs of ghost state and Kani [4, 14] for bounded checking of `unsafe`. The interesting problem is not that a bijection between two maps can be proved. It is that a project shipping a hypervisor will be tempted to put that bijection in the abstract and then spend the rest of the paper taking it back. We treated that temptation as the thing to study.

The proved artifact is exclusivity among guests in a host-only ghost model. Live executable EPT is a bounded ownership registry. Device emulation, HTTP, the SPA, ISO handling, and guest firmware are outside the Proven Core. Claiming "the hypervisor is proved" would be proving the wrong thing [12]: a convenient proxy.

This paper makes three contributions:

1. **A two-axis labelling ladder** (Section 3.4): proof maturity L0–L3 against artifact (ghost model / executable module / binary on the target). An L3 claim about a ghost model and an L3 claim about executable code are not comparable. Table 7, with Section 6.4, shows the field already traversing that axis (seL4 C, then seL4 binary) and resolves a SeKVM abstract that otherwise reads as a proved binary.
2. **Six lessons that generalized** while shipping the EFI (Section 7): pin the prover, keep SMT off the UEFI target, do not poke third-party firmware, firmware TCP is not a management plane, nested VT-x is not

the production server, and derive ghost state from the threat model's domain rather than from the live data structures. Table 8 archives the negative results behind those lessons.

3. **An iron Type-1 with a named isolation gap.** On one R640: Linux 6.12 plus three stub guests, virtio-blk, virtio-net, dual-vCPU SMP, host-NIC HTTP, and El Torito CD EFI. The L3 theorem is a 4K ghost model; production GPA coverage is an L1 range registry (Section 5.1).

Isolon is not seL4 for hypervisors and is not a verified KVM. It is a hypervisor we boot on iron, a ghost model we discharge, and a ladder that stops those two facts from being fused in a sentence.

2 Threat Model and Non-Goals

2.1 Why memory isolation is the headline

A Type-1 hypervisor is the TCB for every guest it runs. CPUID and MSR firewalls, interrupt injection, and the hypercall door all matter. None of them matter if EPT can map the same host frame to two guests without anyone noticing. We therefore name *exclusivity among guests* as the property we prove, not "the guest seemed isolated in tests," and not "the hypervisor never touches a guest frame." On the R640 that property is L1-enforced: hole arithmetic in a 16-slot range table plus runtime checks, currently for Linux plus three stub guests. Sixteen range slots fail-closed on `Full`; that bring-up occupies five rows, so the bound is on the order of a dozen guests, not a cluster. The 512-slot 4K table likewise returns `Full`. Both caps are bring-up artifacts we intend to lift; exhaustion is fail-closed. The L3 theorem is a 4K ghost path that production Linux GPA coverage does not use (Section 5.1).

2.2 In scope

We assume:

- A **malicious or compromised guest** (ring-0 Linux, firmware, or installer) that may issue any architected VM exit: EPT violation, CPUID, MSR, I/O, hypercall, HLT, exception.
- **Buggy device emulation** outside the Proven Core (virtio, IDE/ATAPI, UART). Bugs there can DoS a guest or confuse an installer; they must not become a second EPT mapping of a Proven Core frame.
- **Operator error** (wrong ISO, exhausted USB stick). Audit events record what happened; they do not enlarge the theorem.

Access versus mapping (virtio). Isolon ships virtio-blk and virtio-net. Virtqueue rings live in guest-allocated frames. Hypervisor emulation at VMX root *must* read and write those frames. That is, by construction, a guest-owned frame touched by the hypervisor. Exclusive ownership in this paper is a statement about *EPT mappings*: a frame in *owned* has exactly one guest as EPT owner, and the hypervisor does not install a second EPT entry for that frame in another guest's EPT. Transient, emulation-mediated access from VMX root is not a second owner. If "belongs to neither the hypervisor" is read as "the hypervisor CPU never loads from that frame," the property is false of the product we ship.

DMA and passthrough. The adversary in this section is a malicious ring-0 guest. Such a guest can program DMA if a device is passed through, or if an emulated device can be driven to DMA outside the guest's frames. Isolon's product path has **no device passthrough**. Guest DMA is mediated by emulation executing in the hypervisor's address space. Exclusive ownership is conditional on no passthrough: it is not a theorem against an IOMMU-less device. Unmodelled DMA is listed in Section 2.3. Closing DMA as a lemma needs an IOMMU story we do not have.

We do *not* assume a correct third-party guest firmware. Forcing a foreign firmware's WaitForEvent internals to skip a check produced a NULL-event page fault and a dead loop. Product ISO boot now uses firmware we author. That firmware is **outside** the Proven Core.

2.3 Assumptions, not theorems

- The Intel SDM's VMX/EPT hardware behaves as documented [10].
- The frozen Verus/SMT stack is trusted for the ghost proofs.
- Host physical memory given to the allocator is the memory the machine has.
- Hypervisor frames are those *absent* from the guest ownership registry. We do not prove that the EFI's own page tables never map a guest-owned frame, and we do not prove that a bug cannot install a guest leaf pointing at hypervisor `.text`.
- No device passthrough; guest DMA is emulation-mediated, as above.
- Nested VT-x on a lab server is not a proof about a PowerEdge R640.
- Full x86 ISA / VMCS host-state correctness is inside the Proven Core *budget* and is not at proof-complete maturity beside the EPT crate.

Table 1. Proven Core budget (cap, not an L3 claim per row).

Module	Est. LOC	Why inside
VMX lifecycle	800	VMXON/VMXOFF leave CPU mode
VMCS management	1,500	Host-state corruption
EPT engine	2,000	Isolation mechanism
Frame allocator	1,500	Double-alloc / UAF
vCPU state	1,000	Save/restore leaks
Interrupt injection	800	Guest privilege
Hypercall door	500	Only intentional channel
MSR/CPUID/CR firewalls	1,200	Host feature subversion
Audit-log integrity	600	Tamper-evident trail
IPI confinement	500	Cross-VM IPI
Cap (code + scaffolding)	15,000	—

3 Isolon Architecture

3.1 Single binary

Everything compiles to one UEFI PE/COFF image. Kernel, initrd, Web UI, and schemas are compressed sections, decompressed on first use. The size target is 15 MB with a 20 MB hard limit. There is no host Linux, no package manager, and no config tree the operator must get right before the first serial line.

3.2 Proven Core boundary

Only the modules in Table 1 are in scope for Verus specifications and progressive proofs. Everything else uses Rust's type system, testing, and review. Promotion into the Proven Core requires a written architecture decision; the default answer is no. The hard cap is 15,000 lines including proof scaffolding.

Lived split, as of this submission: the machine-checked exclusivity theorems live in a host-only crate (`ept_model`, 2,366 lines) that is *not* linked into the `.efi`. The executable EPT ownership registry is 705 lines at spec-written maturity. Device emulation, HTTP, the SPA, ISO, guest firmware, iDRAC, and migration tooling are outside.

3.3 Hardware

Dell PowerEdge R640 / R650 / R660. Low-risk, self-sufficient surfaces (serial via iDRAC virtual console, SMBIOS/ACPI topology, onboard LOM) are in scope. OEM RAID health schemas are not a blocker for the theorems or for the single-host product loop.

Code in the Proven Core is required to document invariants at the function boundary, keep `unsafe` blocks minimal, and tag each of them for bounded checking. Mutable globals require a lock and a justification comment. We use explicit state machines rather than boolean flags. These rules are

Table 2. Isolon on two axes. Empty cells are not claimed.

	Ghost model	Executable module	Binary on target
L3	`ept_model` exclusivity	—	—
L2	—	`memory/ept.rs`	—
L1	—	Kani on `unsafe`	R640 Linux+virtio+SMP

how a codebase stays L3-shaped when most modules are still L1: the later proof should not require a rewrite of ownership.

3.4 Maturity ladder as a community instrument

Proof maturity is one axis:

- **L0** documented invariants in comments.
- **L1** runtime `assert!`/`debug_assert!` plus Kani on `unsafe`.
- **L2** Verus specifications with ghost state and contracts.
- **L3** `cargo verus verify` succeeds for the claimed surface.

The second axis is *which artifact* the claim is about: a ghost model, an executable module, or a binary on the target. Table 2 places Isolon; Table 7 scores the same cells beside seven prior systems. An L3 claim about a ghost model and an L3 claim about executable code are not comparable; a one-axis label "L3 isolation" would make Isolon's crate look like seL4's C kernel.

Fallback is Verus → Kani → asserts and fuzz. Shipping is allowed at L1/L2 when L3 is still open, provided the paper names the cell.

Proven Core modules follow a four-file layout: executable Rust, a spec file, a proof file, and tests (Kani + unit). The exclusivity theorems of record do not use that layout. They live in one `verus!` block in `ept_model`. Putting them beside live `memory/ept.rs` would imply the EFI crate is L3. It is not.

4 Exclusive Ownership

4.1 Statement

> For every valid EPT mapping from a guest-physical address to a host-physical frame, that frame is owned by exactly one guest: no other guest holds a mapping to it.

"Exclusively owned" means exactly one guest holds a mapping to that frame at any moment. The property must hold across map, unmap, EPT-violation handling, and live-migration page transfer.

Hypervisor separation—the informal sentence "belongs to neither the hypervisor nor any other guest"—is *not* this

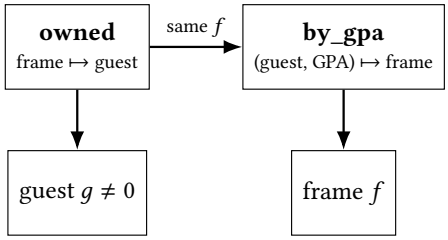

Figure 1. Ghost EPT ownership registry. Exclusive ownership is four conjuncts: every owned frame has a matching by_gpa witness; every by_gpa entry has a matching owner; a frame is the image of at most one pair; the two maps have equal length.

theorem. Guest id 0 is reserved; hypervisor frames are modelled as absent from *owned*. Absence is definitional. We list it in Section 2.3 next to the SDM and the SMT stack. The ghost crate never mentions the EFI's own page tables. A bug that installed a guest leaf pointing at `.text` would not be caught by these lemmas.

4.2 Ghost state

The ghost model is two finite maps (Figure 1):

- $owned : FrameId \rightharpoonup GuestId$
- $by\_gpa : (GuestId \times Gpa) \rightharpoonup FrameId$

Guest id 0 is reserved (hypervisor / unmapped). 4K steps require page-aligned GPAs. A map step is enabled only when the frame is free and the (guest, GPA) pair is free. An unmap step is enabled only when that pair is present.

The predicate (from the crate, names only) is:

> exclusive_ownership(m) iff every owned frame has a by_gpa witness, every by_gpa entry has a matching owner, a frame is the image of at most one pair, and the two maps have equal length.

Empty maps satisfy it. A hypervisor frame is modelled as *not* being in *owned*: the ghost registry only tracks guest-held frames.

4.3 Steps

A 4K step is $\mathsf{Map}\{g, a, f\}$ or $\mathsf{Unmap}\{g, a\}$. Enabled steps preserve exclusive ownership (one-step lemma). Any finite sequence of enabled steps preserves it. Guest ids are already a field of each step, so the sequence theorem is already an N-guest statement. A two-guest lemma maps two distinct non-zero guests onto distinct free frames.

Large pages are spans of 4K frames (512 for 2M, 262,144 for 1G). Span map/unmap lemmas lift exclusivity to 2M/1G. Hardware 2M-leaf decode lemmas relate a leaf PTE encoding to an identity-map intent. A ghost SRAT/SLIT records which node a mapped frame sits on; SLIT distance never appears in the four exclusivity conjuncts, so affinity is orthogonal bookkeeping, not part of the isolation theorem. None of these lemmas are a full multi-level EPT walk of the EFI.

Table 3. Named exclusivity theorems in the host-only crate.

Theorem	Claim
`single_guest_4k_map_unmap`	finite 4K sequences
`n_guest_4k_map_unmap`	CI alias of the previous row
`concrete_*_guest_refine`	scoped ghost↔exec
`large_page_map_unmap`	2M/1G spans
`numa_map_unmap_affinity`	node affinity (orthogonal)
`hw_2m_leaf_refines_identity`	2M leaf encode
`ept_violation_preserves`	miss dispositions
`page_transfer_preserves`	unmap→map handoff
`alloc_map_unmap_refines`	pool + EPT refine

4.4 EPT-violation dispositions

An EPT miss is modelled as one of three dispositions:

- EmulateNoMap — emulate MMIO (APIC / virtio) without installing a mapping. Registry unchanged.
- Reject — fail closed. Registry unchanged.
- $\mathsf{ClaimMap}\{g, a, f\}$ — demand-fill a 4K map, enabled only when that map step would be enabled.

Every enabled disposition preserves exclusive ownership. Emulate-then-claim is a derived post.

4.5 Page transfer

Live migration of a frame is unmap-at-source then map-the-same-HPA at destination. The step is enabled when source holds the GPA, destination GPA is free, and guests are distinct and non-zero. Posts: destination owns the frame at the new GPA; the source mapping is gone; exclusivity holds. This is a ghost primitive. It is not vCenter vMotion, not a multi-host protocol, and not a proof that the product migrate engine is correct. That engine is outside the Proven Core.

4.6 Theorems of record

Table 3 lists the named theorems. Supporting lemmas (empty-exclusive, map-already-owned-unchanged, span preservation, mock bring-up facts) are in the same crate. The 80/0 CI line is in Table 4.

4.7 Proof sketch: map preserves exclusivity

We sketch the 4K map case; unmap is the inverse bookkeeping. Assume $exclusive\_ownership(m)$, $g \neq 0$, a 4K-aligned,

$f \notin \mathrm{dom}(owned)$, and $(g, a) \notin \mathrm{dom}(by\_gpa)$. Let $m' = m.\mathrm{ghost\_map}(g, a, f)$.

Witness. For the new frame f, the pair (g, a) is the witness required by conjunct 1. Old frames keep their old witnesses because we did not delete by_gpa entries.

Owner match. The new by_gpa entry points at f, and $owned'[f] = g$. Old pairs are unchanged.

Uniqueness. Suppose two pairs in m' share a frame. If that frame is f, both pairs must be (g, a) because f was free. If not, uniqueness in m applies.

Cardinalities. Both maps gain one key, so lengths stay equal.

Verus discharges this with map lemmas from vstd (broadcast group_map_lemmas / group_set_lemmas). The one-step lemma case-splits on Map vs Unmap. The sequence theorem is induction on Seq length: the empty sequence is trivial; the inductive step applies the one-step lemma and recurses. The N-guest CI name is that same induction.

4.8 What the obvious formulation gets wrong

Violation dispositions. The miss handler is where every hypervisor decides whether a GPA is RAM or a device. Demand-fill—every EPT violation becomes ClaimMap—is the natural first implementation: the guest touched it, so map it. On Isolon that would install a leaf for APIC and virtio MMIO GPAs, either aliasing device memory into the ownership registry or handing the guest a host frame for a region that should stay unmapped. A later guest that maps the same HPA would then see AlreadyOwned for a frame that should never have been guest RAM. The trichotomy (EmulateNoMap / Reject / ClaimMap) is the systems fix, independent of Verus: classify the GPA before installing a leaf. MMIO is emulated with the registry unchanged; fail-closed is identity on the maps; demand-fill is enabled only when the 4K map step would be. Emulate-then-claim is a derived post. Any EPT/NPT miss path has the same fork. The bug class is missing the fork, not a missing SMT lemma.

2M/1G spans. The obvious encoding of a 2M map is a loop of 512 4K maps. That is the wrong object. SMT would need a loop invariant over 512 iterations, still would not know that those 512 keys are *one* leaf with the page-size bit, and would still be 512 ghost steps rather than one hardware PDE. What we wrote instead is a span predicate (every aligned 4K key in the region is free) followed by a bulk insert, plus an arithmetic lemma that 512 aligned 4K frames constitute a 2M leaf (262,144 for 1G). Those facts are still not a hardware page-walk. They are why "map 4K k times" is not the 2M theorem.

SMT as bug finder. We did not catch an executable aliasing bug with Verus or Kani. Section 7.6 is why. Runtime on iron found real bugs—Enable-XSAVES, INVPCID, WaitForEvent state pokes (Table 8)—none of which are exclusivity lemmas.

Violation ClaimMap reuses the 4K map lemma. Emulate and reject are identity on the registry; the proof is $m' = m$. Transfer is unmap then map of the *same* frame: after unmap, f is free, so the destination map is enabled. Posts are the destination owner and the absent source key.

5 The Ghost / Executable Gap

A discharged ghost model is not a proved .efi.

Live EPT (memory/ept.rs, 705 lines) maintains an ownership registry and asserts exclusive-ownership-shaped facts at map/unmap. Beside it sit 357 lines of spec/proof notes. Those notes are not the 80-function crate. The host-only crate exists *because* linking Verus into the UEFI target is the wrong packaging: proofs run on a Linux CI host under a pinned Verus release; the EFI is a UEFI target, not a Linux sysroot.

Scoped refinement lemmas show that a concrete map representation, under a restricted interpretation, commutes with ghost map/unmap for 4K (one guest, then N guests) and that an allocate-then-map path refines. They are not a bisimulation of VMXON, VMCS host state, EPT pointer load, and every MMIO emulator the EFI contains.

Consequences:

- L3 is the ghost crate. Live EPT is L2.
- VMX, VMCS, hypercalls, MSR firewalls, audit integrity, and IPI confinement are in the budget in Table 1 and are not at L3 beside ept_model.
- Iron serial logs raise confidence in the *runtime path*. They do not raise Verus maturity. Lab L3 and R640 L1 are both real; they are not the same claim.

5.1 Live registry versus ghost maps

The live engine is not a Verus Map. It is a bounded table: EptMap holds at most 512 explicit 4K claims in production builds (8 under Kani, so CBMC can unwind), plus a 16-slot range registry for the precise identity window used at Linux bring-up.

Collision and exhaustion are different errors. Map or range-claim of a taken frame or overlapping span returns AlreadyOwned. When every slot is occupied, both tables return Full. There is no silent drop and no fallback from 4K slots into range claims.

Linux bring-up does not fill the 512-slot 4K table. Guest e820 is 256 MiB; the precise identity window is [0, PRECISE_BYTES) with PRECISE_BYTES=512 MiB

(131,072 4K pages). The window matches QEMU `-m 512M`: it covers guest page tables, the early frame pool, and 256 MiB of guest RAM, while leaving the local APIC at `0xFEE00000` unmapped by omission. The upper 256 MiB is the BAR and shell window: virtio BARs and G1–G3 2M holes sit above e820 and inside the 512 MiB identity map. Hardware identity uses 2M leaves (256 leaves).

The executable counterpart of that window is the range registry, not the proved 4K table. With no extra-guest holes, bring-up inserts **one** range covering the full 512 MiB for the Linux guest (host test: `EptRangeMap::len() = 1`). Multi-VM probes punch private HPA holes above e820 and claim those spans for shell guests. Linux plus three stubs occupies five range rows: G0 below e820, three 2M shells, G0 remainder (host test: `len()=5`). Pairwise HPA non-overlap on that path is `claim_range` returning `AlreadyOwned` on overlapping spans, plus a sorted-hole cursor that rejects `hpa < cursor` as `Invariant`. After insert, the boot path checks that G0 owns GPA 0 and the last e820 page, and that each hole's guest owns its base while G0 does not. The host test additionally checks that G1 does not own G2's base, G2 does not own G3's, and G3 does not own G1's. It does not enumerate every guest-pair of the 512-slot 4K table. The 4K self-test is a *different* object: two-guest alias on `EptMap`. Iron exclusivity among Linux and the three stubs is therefore hole arithmetic in the 16-slot table, L1-tested, not a ghost lemma.

Scoped refinement lemmas interpret the 4K table into ghost maps. They do not interpret the range registry. Production guest-physical coverage at Linux boot—the 512 MiB identity window, including the 256 MiB e820—goes through the range path. That path has no correspondence in the proved model. This is not a gap between two encodings of the same mapping mechanism; it is disjointness. The 4K table is the demand-fill and self-test path (map a bring-up guest, then assert a second guest cannot alias the same frame), including MMIO-not-mapped misses that take ClaimMap. If a 513th distinct 4K claim is required, the live table returns `Full` while the ghost theorem still holds of an abstract `ghost_map`. That is why Table 2 leaves the executable-module L3 cell empty.

A range claim is a span. The crate already has span predicates for 2M/1G. Modelling a range row as a ghost step, with pairwise HPA non-overlap among the 16 slots and against 4K rows, is the lemma that would put the production path in the same artifact column as the proof. This paper does not claim that lemma.

5.2 What would close the gap

Four artifacts, none of which this paper claims:

1. A ghost range-claim step whose posts are pairwise HPA non-overlap among range rows and against 4K rows, so the Linux window is in the proved model.
2. A proved executable EPT walker whose leaves are in 1–1 correspondence with `by_gpa`, including the HV-vs-guest split ("belongs to neither" as a fact about HV page tables, not as absence from a guest registry).
3. An IOMMU / DMA story: devices we do not model cannot write a guest-owned frame or a hypervisor frame.
4. Linking or replaying (2) on the UEFI target, or a refinement from the EFI binary's EPT root pointer to the ghost maps.

Until then, L3 names the ghost crate. The EFI is not L3.

6 Evaluation

We evaluate five questions. (1) Does the ghost model verify? (2) How large is the proved surface? (3) What covers guest-physical space at Linux boot? (4) What does the runtime do on a lab host versus a real R640? (5) Does the two-axis ladder resolve abstract-level ambiguity in published systems?

6.1 Proofs

Toolchain: Verus pin `0.2026.07.12` (asset `0b42f4c`; Linux zip SHA-256 `f6f4f5d0. . . fbd01c0` in-tree; CI never uses `latest`) and Kani 0.67.0 (CBMC backend). The exclusivity crate is verified with `cargo verus verify`. Against that pin the crate reports **80 verified, 0 errors**, including the N-guest alias, with no `admit` on the exclusivity path. Wall-clock and peak RSS for that CI run were not logged (Section 9). Kani covers bounded `unsafe` in the live engine (`MAP_CAP = 8` unwind). That is L1 on `unsafe`, not L3 on exclusive ownership.

6.2 Size and occupancy

Table 4 is tree- and kit-measured. The iron EFI that reached Linux shell plus the M4 probe chain is 1,085,952 bytes (1.04 MiB), against a 15 MB target and a 20 MB hard limit. A later in-tree release build is 1.62 MiB. Rust sources in the tree total 64,140 lines (`wc -l` of `*.rs`, excluding `target/`): about 40,500 product, 15,600 gate harnesses, 8,000 tests. The Proven Core *budget* remains 15,000 lines (Table 1); the surface at L3 is the 2,366-line host-only crate. Table 5 is the EPT slice.

Range occupancy is from the host test of the same `claim_precise_*` function the boot path calls. Linux-only: one range row covering 512 MiB. Linux plus three stubs: five rows (G0 below e820, three 2M shells, G0 remainder). That occupancy is evaluation question 3—what covers guest-physical space at boot—and the domain Lesson 6 says

Table 4. Measured quantities. Dashes: R640 not re-instrumented this cycle; Verus wall-clock not logged on CI (Section 9).

Quantity	Value
Iron `.efi` (Linux+M4 boot)	1,085,952 B (1.04 MiB)
In-tree release `.efi`	1.62 MiB
Size target / hard limit	15 MB / 20 MB
Tree Rust LOC (`*.rs`)	64,140
Proven Core budget / L3 crate	15,000 / 2,366
Verus pin zip SHA-256	`f6f4f5d0. . . fbd01c0`
Verus functions (incl. alias)	80 verified, 0 errors
Linux-only range occupancy	1 row (host test)
Linux+3 stubs range occupancy	5 rows (host test)
Hardware 2M leaves in the window	256 (131,072 4K pages)
Guest e820 RAM / identity window	256 MiB / 512 MiB
4K table cap / range slots	512 / 16; exhaustion = `Full`
VM-exit cycles vs KVM on R640	—
Boot-to-shell wall-clock vs KVM	—
`verus verify` wall-clock / RSS	—

Table 5. Line counts (`wc -l`) for the EPT-related surface.

Path	Lines	Role
`ept_model` crate	2,366	L3 ghost + proofs
`memory/ept.rs`	705	live EPT (L2)
`ept_spec` + `ept_proof`	357	notes beside live
frame allocator + spec	362	allocator + ghost set

to model first. Hardware uses 256×2M leaves. Guest e820 is 256 MiB. The R640 was not re-instrumented this cycle, so there is no COM2 slot-counter dump and no VM-exit cycle comparison (Section 9).

6.3 Runtime: lab host versus R640

Table 6 is the split. Cells are contemporaneous serial or host-test evidence, not a wish list. Nested VT-x is a third column because operators (and reviewers) otherwise collapse it into iron.

On that R640 (iDRAC virtual media, COM2): the EFI archives VMXON, a 512 MiB identity window, unmodified Linux 6.12 through `/init` to a shell, then—on that same boot—four-VM probes (Linux G0 plus three SHELL guests), virtio-blk, virtio-net, dual-vCPU SMP, and VMXOFF. Secondary Enable-XSAVES was required on Xeon Scalable; without it the guest init hit a stack guard. Firmware SNP/Tcp4 after ExitBootServices is a platform limit of the virtual-floppy path; durable HTTP is a host-owned Broadcom LOM. The SPA can VMLAUNCH a private-EPT SHELL guest and re-enter; that guest is a CPUID stub, not a distro. Guest firmware issued ATAPI `READ(10)` and booted El Torito CD EFI. Persist-detect stamps on a USB stick are LBAs, not a guest root filesystem.

On nested VT-x we authored guest firmware after third-party WaitForEvent forcing faulted. That firmware completed an alpine-extended `setup-disk -m sys` (installer printed "Please reboot") and reboot-to-disk. Nested virtualization is not the R640. The USB stick used on iron is too small for that distro image. Host CI and nested runs do not print an install-complete marker.

6.4 Labelling the literature

The grid does two things a one-axis "L3 isolation" label cannot. It resolves abstract-level ambiguity in published systems, and it keeps three Isolon claims from fusing (Table 7, last three rows). Table 7 is a reading of each paper's abstract against the stated artifact, not a re-proof.

seL4 already walked the artifact axis. The 2009 seL4 abstract names the C implementation of the kernel [13]: cell (L3, executable C). Sewell, Myreen, and Klein [22] then closed C-to-binary on ARM by translation validation: cell (L3, binary on the target). The field spent four years treating that distinction as worth a separate PLDI paper. It did not have a name for the axis. The ladder is that vocabulary.

SeKVM's abstract is the detection. SeKVM's abstract presents "a formally verified Linux KVM hypervisor" [18, 19]. That sentence reads as cell (L3, binary on the target). The Coq development is an overlay covering a module of commodity KVM: cell (L3, executable module). The grid does not need to accuse anyone of anything. It names the two cells so a reader can see they are not the same object.

This paper's own drafts. An earlier draft of this submission put "belongs to neither the hypervisor nor any other guest" in the abstract, next to a live ownership registry, with the R640 boot in the same paragraph. Under the grid that paragraph claims cell (L3, binary on the target). The evidence was cell (L3, ghost model). The instrument would have split the paragraph. That is the evaluation nobody else can run: the same project, before and after the label.

XMHF and Verisoft XT name fragments of an executable hypervisor, not a whole binary [17, 26]. CertiKOS, Komodo, and Hyperkernel name an executable artifact and the evidence matches [5, 7, 21].

Table 6. Split evaluation. Nested VT-x ≠ PowerEdge. Host Verus ≠ EFI. The Lab/CI column is where the prover runs; the R640 column is the only iron.

Surface	Lab / CI	Nested VT-x	PowerEdge R640
Verus exclusivity (80 / 0)	yes (L3)	—	not claimed
Kani on `unsafe`	yes (CI pin)	—	not claimed
VMXON + VMEXIT	yes	yes	yes
EPT identity + Linux 6.12 shell	yes	—	yes
LAPIC timer re-entry / IRQ inject	yes	—	yes (same boot)
virtio-blk probe	yes	CD+virtio	yes (same boot)
virtio-net + vSwitch probe	yes	—	yes (same boot)
Dual-vCPU BSP+AP (shared EPT)	yes	—	yes (same boot)
Four VMs: Linux + three SHELL stubs	yes	—	yes (same boot)
Durable host-NIC HTTP after EBS	lab NIC	—	onboard LOM
SPA launches a SHELL guest	host smoke	—	stub, not a distro
Guest firmware CD EFI (El Torito)	—	CD+virtio	yes
Distro sysinstall + reboot-to-disk	—	yes	open
Persist-detect LBA stamps on USB	—	—	stamps, not a rootfs

Table 7. Retrospective (maturity, artifact) reading. The two seL4 rows are one kernel, four years apart. The three Isolon rows are one system whose cells a one-axis label would fuse.

System	Mat.	Abstract reads as	Evidence artifact
seL4 (2009 C) [13]	L3	executable C kernel	executable C
seL4 (2013 binary, same kernel) [22]	L3	ARM binary	ARM binary
CertiKOS [7]	L3	C implementation	executable C
Komodo [5]	L3	monitor implementation	executable monitor
Hyperkernel [21]	L3	simplified kernel	executable kernel
SeKVM [19]	L3	Linux KVM hypervisor	executable overlay
XMHF [26]	mixed	hypervisor framework	executable fragments
Verisoft XT [17]	mixed	Hyper-V	selected C (VCC)
Isolon exclusivity	L3	ghost exclusivity crate	ghost model
Isolon `memory/ept.rs`	L2	—	executable module
Isolon R640 boot	L1	Linux + 3 stubs on iron	binary on target

6.5 Artifact

We commit to a supplementary zip: the `ept_model` crate, the pinned Verus zip SHA-256, a README that reproduces 80/0, and a container or exact install command for that pin. Anonymized COM2 excerpts for every row of Table 6's R640 column (product markers stripped) belong in the same zip. The paper stands alone if the reviewer ignores the zip.

7 Experience: Six Lessons that Generalized

The following are the lessons we would tell a group starting a verified Type-1 hypervisor in Rust. They are the spine of the paper.

7.1 Pin the prover or the proof is a diary

Verus and its SMT backend move. We froze the tagged Verus release `0.2026.07.12` (asset `0b42f4c`) with a SHA-256 of the Linux zip in-tree, and forbade CI from fetching `latest`. The 80/0 line is replayable only against that pin. Groups that verify "on nightly" will not be able to say, a quarter later, whether a new `admit` is a regression or a toolchain change. Budget pin maintenance as a first-class task, not as hope.

7.2 The UEFI target cannot run SMT

Isolon's EFI target is UEFI PE/COFF, not a Linux sysroot. Verus wants a host sysroot. We stopped trying to link proofs into the PE/COFF image. The 2,366-line crate lives on the CI host; the 705-line engine lives in the EFI. That split is why L3 and L2 are different columns in Table 2. A single-crate

"verified hypervisor" that does not build on the target you ship will still be described as verified by someone; make the packaging match the claim first.

7.3 Do not poke third-party firmware

When guest firmware blocked in WaitForEvent, a tempting fix is to poke its wait object, skip a check, or inject a fault. We did that on a development path. The firmware then page-faulted on a NULL event pointer and entered a dead loop. We do not own that firmware's invariants; forcing one of them violates the next. The product path is firmware we author, driven only through architected inputs (interrupts, device registers, DMA the spec says is ours). Device emulation remains outside the Proven Core, so a firmware bug is still not an EPT exclusivity theorem—but it is no longer a self-inflicted one. This lesson is independent of Isolon: if you do not own the blob, do not rewrite its control state.

7.4 Firmware TCP is not a management plane

iDRAC virtual floppy advertised PXE/HTTP but not a Tcp4 service binding. Firmware SNP that had accepted a listen before ExitBootServices did not survive it. Durable HTTP on the R640 is a host-owned Broadcom LOM after the hypervisor owns the NIC, not a firmware socket. Quantitative split: PRE-EBS HTTP closed on SNP as a bring-up window; post-EBS HTTP closed only after a native driver. Papers that claim "the hypervisor serves a Web UI" should say which NIC stack and which side of ExitBootServices. In-process REST on a lab executable is a third, still weaker, claim.

7.5 Nested VT-x is not the production server

Nested alpine-extended sysinstall completed: installer printed "Please reboot," then reboot-to-disk succeeded. That is a real product-path result on nested VT-x. The USB stick used on iron is too small for that distro image; El Torito CD EFI is not `setup-disk`. A lab hypervisor that installs Linux under nested VT-x has not demonstrated ISO install on the machine in the rack. Table 6 keeps those columns apart.

7.6 Derive ghost state from the threat model's domain

Isolon's ghost exclusivity model was written against `EptMap`, the 512-slot 4K table that already returned `AlreadyOwned` on a second map of the same HPA. SMT then found no mapping bug, because the model encodes the check the code already had. The blind spot was not random: production guest-physical coverage is mostly range rows, and those never entered the ghost state.

Derive the ghost state from the threat model's domain—here, every guest-physical frame the hypervisor may map—not from the implementation's data structures, then check which code paths the model fails to cover. Had we enumerated "what covers guest-physical space at boot" before choosing what to model (evaluation question 3), the range registry would have been in the crate from the start. Anyone retrofitting a proof onto a working system should run that coverage check first.

7.7 Negative results we archived

Table 8 lists failures we kept. None of them are L3.

Table 8. Archived negative results. None of these are L3.

Failure	What it blocked
Guest `/init` stack guard without Enable-XSAVES	Linux shell on Xeon Scalable
Firmware Tcp4 absent on virtual floppy	"HTTP forever" via SNP
SNP listen dead after ExitBootServices	post-EBS firmware UI
WaitForEvent state poke → #PF	third-party firmware as product ISO path
USB persist LBA stamps	calling stamps a root filesystem
USB stick capacity < alpine-extended	iron sysinstall on that stick

The XSAVES miss is the most transferable: a VMCS secondary-control bit that is optional on a laptop CPU and required on the server. Isolation proofs do not mention it. The guest still does not boot. Treat CPU-feature bring-up as L1 evidence, not as a corollary of exclusive ownership.

8 Related Work

Verified systems and their unverified edges. Fonseca et al. [6] audited IronFleet, Verdi, and Chapar from the outside and found 16 bugs, not in protocol logic but at the interface of verified and unverified components—specifications, shims, and tools. That is this paper's thesis, at this venue, from the other direction: Isolon reports the same class of gap from inside one Type-1 hypervisor and proposes a (maturity, artifact) label so an abstract cannot place an L3 overlay on a commodity binary, or an L3 ghost model on the EFI that left the compiler. Table 7 is the evaluation of that instrument.

Proved kernels and enclaves. seL4 [12, 13, 20] is the landmark machine-checked microkernel. The seL4 team was careful about proof assumptions—hardware, compiler, assembly, boot—and later closed the C-to-binary gap on ARM by translation validation [22]. Isolon borrows the discipline and a fraction of the scope: we do not prove the scheduler or the device model, and we do not have a binary-level theorem. CertiKOS [3, 7] builds a certified concurrent kernel with contextual refinement. Komodo [5] verifies a software enclave

monitor on ARM TrustZone. Ironclad Apps [8] push verification out to application principals. These systems show isolation *can* be proved. They do not ship as `BOOTX64.EFI` on iDRAC virtual media.

Hypervisors with a verification story. Hyperkernel [21] pushes a hypervisor toward push-button verification with a simplified interface. XMHF [26] is an extensible hypervisor framework with selected proofs. Verisoft XT applied VCC to Hyper-V components [17]. SeKVM / MicroV work [18, 19] layers formal memory protection onto a commodity KVM. Isolon is the opposite packaging: a new UEFI binary with a tiny proved ghost registry, not a verified overlay on Linux.

Small-TCB virtualization. NOVA [24] and related microhypervisor designs argue for a small TCB in the EuroSys tradition. Isolon's cut is the Proven Core list in Table 1. OKL4-style microvisors [9] similarly shrink the host. Production hypervisors—ESXi [2], Xen [1], KVM [11]—isolate with EPT/NPT and years of testing.

Tooling. Verus [15, 16] is the SMT prover we pin. Kani [4, 14] covers `unsafe`; it compiles MIR to CBMC. VanHattum et al. [25] is the ICSE-SEIP paper on dynamic trait objects in that pipeline, not a substitute for the tool or for CBMC. Push-button verification of file systems [23] is the methodological cousin of Hyperkernel.

9 Limitations

- Live executable EPT is L2. L3 is the host-only ghost model. Ghost↔exec refine is scoped, not a full EFI bisimulation. The range registry that covers Linux GPA has no ghost correspondence.
- Most Proven Core modules in Table 1 are not at L3.
- Large-page and NUMA L3 are lab/CI. Iron stopped at Linux plus three stub guests for those surfaces.
- `PageTransferStep` is a ghost primitive, not live migration product.
- DMA confinement and IOMMU are future work. Exclusive ownership is conditional on no passthrough and emulation-mediated DMA.
- Nested distro sysinstall is complete and is not iron.
- Lab soak, audit-report templates, and a bring-up spec review are L0/L1 process gates, not L3. Iron distro install is a product gate, not a requirement of the ghost theorem.
- Iron installer, TLS, and guest console remain product residuals.
- Verus upgrades can break proofs; pin maintenance forbids `latest`.
- SMT found no executable aliasing bug (Section 7.6).
- No KVM cycle comparison or `verus verify` wall-clock.

Threats to validity. Iron evidence is one PowerEdge R640 and iDRAC virtual media, not a fleet. Nested VT-x is one lab host. The 80/0 line is one pin of one prover; we have not replayed under a second SMT solver. LOC counts are `wc -l`, not a proven TCB. Range occupancy is a host test of the boot claim function. The R640 was not available for re-instrumentation during this submission cycle, so VM-exit cycle counts, a COM2 slot-counter dump, and a KVM comparison are absent. The 80/0 CI run predates this submission and was not instrumented for timing. Table 7 is our reading of published abstracts, not a re-verification of those systems. Experience lessons (XSAVES, SNP, firmware pokes) are one implementation's failures; they follow from Intel VMX, UEFI ExitBootServices, and "do not own that blob," not from Isolon-specific APIs.

10 Conclusion

Isolon is a UEFI Type-1 hypervisor that boots unmodified Linux 6.12 with virtio and SMP on a PowerEdge R640, and an experience report about the gap between that binary and a machine-checked isolation claim. The two-axis ladder is the instrument: it names the axis seL4 already walked from C to binary, and it splits a SeKVM abstract that otherwise reads as a proved hypervisor. The 512 MiB Linux window is a range registry with no ghost correspondence. Nested authored firmware completed a distro sysinstall. Nested is not iron.

References

[1] Paul Barham, Boris Dragovic, Keir Fraser, Steven Hand, Tim Harris, Alex Ho, Rolf Neugebauer, Ian Pratt, and Andrew Warfield. 2003. Xen and the Art of Virtualization. In *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP)*. 164–177. https://doi.org/10.1145/945445.945462

[2] Edouard Bugnion, Scott Devine, Mendel Rosenblum, Jeremy Sugerman, and Edward M. Bugnion. 2012. Bringing Virtualization to the x86 Architecture with the Original VMware Workstation. *ACM Transactions on Computer Systems* 30, 4 (2012), 12:1–12:51. https://doi.org/10.1145/2384592.2384593

[3] David Costanzo, Zhong Shao, and Ronghui Gu. 2016. End-to-End Verification of Information-Flow Security for C and Assembly Programs. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 648–664. https://doi.org/10.1145/2908080.2908100

[4] Rémi Delmas, Zyad Hassan, Qinheping Hu, Rahul Kumar, Felipe R. Monteiro, Thanh Nguyen, Adrián Palacios, Celina Val, Michael Tautschnig, Justus Adam, Daniel Schwartz-Narbonne, and Carolyn Zech. 2026. Kani: A Model Checker for Rust. arXiv:2607.01504. To appear, 39th IEEE/ACM International Conference on Automated Software Engineering (ASE).

[5] Andrew Ferraiuolo, Andrew Baumann, Chris Hawblitzel, and Bryan Parno. 2017. Komodo: Using Verification to Disentangle Secure-Enclave Hardware from Software. In *Proceedings of the 26th ACM Symposium on Operating Systems Principles (SOSP)*. 287–305. https://doi.org/10.1145/3132747.3132782

[6] Pedro Fonseca, Kaiyuan Zhang, Xi Wang, and Arvind Krishnamurthy. 2017. An Empirical Study on the Correctness of Formally Verified Distributed Systems. In *Proceedings of the Twelfth European Conference on Computer Systems (EuroSys)*. 16 pages. https://doi.org/10.1145/3064176.3064183

[7] Ronghui Gu, Zhong Shao, Hao Chen, Xiongnan (Newman) Wu, Jieung Kim, Vilhelm Sjöberg, and David Costanzo. 2016. CertiKOS: An Extensible Architecture for Building Certified Concurrent OS Kernels. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 653–669.

[8] Chris Hawblitzel, Jon Howell, Jacob R. Lorch, Arjun Narayan, Bryan Parno, Danfeng Zhang, and Brian Zill. 2013. Ironclad Apps: End-to-End Security via Automated Full-System Verification. In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 165–181.

[9] Gernot Heiser and Ben Leslie. 2010. The OKL4 Microvisor: Convergence Point of Microkernels and Hypervisors. In *Proceedings of the First ACM Asia-Pacific Workshop on Systems (APSys)*. 19–24. https://doi.org/10.1145/1851276.1851282

[10] Intel Corporation. 2024. Intel 64 and IA-32 Architectures Software Developer's Manual, Volume 3: System Programming Guide. (2024). Order number 325384. Extended page tables (EPT) and VMX.

[11] Avi Kivity, Yaniv Kamay, Dor Laor, Uri Lublin, and Anthony Liguori. 2007. KVM: The Linux Virtual Machine Monitor. In *Proceedings of the Linux Symposium*. 225–230.

[12] Gerwin Klein, June Andronick, Kevin Elphinstone, Toby Murray, Thomas Sewell, Rafal Kolanski, and Gernot Heiser. 2014. Comprehensive Formal Verification of an OS Microkernel. *ACM Transactions on Computer Systems* 32, 1 (2014), 2:1–2:70. https://doi.org/10.1145/2560537

[13] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. 2009. seL4: Formal Verification of an OS Kernel. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles (SOSP)*. 207–220. https://doi.org/10.1145/1629575.1629596

[14] Daniel Kroening and Michael Tautschnig. 2014. CBMC – C Bounded Model Checker. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. 389–391. https://doi.org/10.1007/978-3-642-54862-8_26

[15] Andrea Lattuada, Travis Hance, Jay Bosamiya, Matthias Brun, Chanhee Cho, Hayley LeBlanc, Pranav Srinivasan, Reto Achermann, Tej Chajed, Chris Hawblitzel, Jon Howell, Jacob R. Lorch, Oded Padon, and Bryan Parno. 2024. Verus: A Practical Foundation for Systems Verification. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP)*. 438–454. https://doi.org/10.1145/3694715.3695952

[16] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. 2023. Verus: Verifying Rust Programs using Linear Ghost Types. In *Proceedings of the ACM on Programming Languages*, Vol. 7. 85:1–85:30. https://doi.org/10.1145/3586037

[17] Dirk Leinenbach and Thomas Santen. 2009. Verifying the Microsoft Hyper-V Hypervisor with VCC. In *Proceedings of the 2nd World Congress on Formal Methods (FM)*. 806–809. https://doi.org/10.1007/978-3-642-05089-5_51

[18] Shih-Wei Li, Xupeng Li, Ronghui Gu, Jason Nieh, and John Zhuang Hui. 2021. Formally Verified Memory Protection for a Commodity Multiprocessor Hypervisor. In *Proceedings of the 30th USENIX Security Symposium*. 3953–3970.

[19] Shih-Wei Li, Xupeng Li, Ronghui Gu, Jason Nieh, and John Zhuang Hui. 2021. A Secure and Formally Verified Linux KVM Hypervisor. In *Proceedings of the 2021 IEEE Symposium on Security and Privacy (SP)*. 1782–1799. https://doi.org/10.1109/SP40001.2021.00049

[20] Toby Murray, Daniel Matichuk, Matthew Brassil, Peter Gammie, Timothy Bourke, Sean Seefried, Corey Lewis, Xin Gao, and Gerwin Klein. 2013. seL4: From General Purpose to a Proof of Information Flow Enforcement. In *Proceedings of the 2013 IEEE Symposium on Security and Privacy*. 415–429. https://doi.org/10.1109/SP.2013.35

[21] Luke Nelson, Helgi Sigurbjarnarson, Kaiyuan Zhang, Dylan Johnson, James Bornholt, Emina Torlak, and Xi Wang. 2017. Hyperkernel: Push-Button Verification of an OS Kernel. In *Proceedings of the 26th ACM Symposium on Operating Systems Principles (SOSP)*. 252–269. https://doi.org/10.1145/3132747.3132748

[22] Thomas Sewell, Magnus O. Myreen, and Gerwin Klein. 2013. Translation Validation for a Verified OS Kernel. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 471–482. https://doi.org/10.1145/2491956.2462183

[23] Helgi Sigurbjarnarson, James Bornholt, Emina Torlak, and Xi Wang. 2016. Push-Button Verification of File Systems via Crash Refinement. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 1–16.

[24] Udo Steinberg and Bernhard Kauer. 2010. NOVA: A Microhypervisor-Based Secure Virtualization Architecture. In *Proceedings of the 5th European Conference on Computer Systems (EuroSys)*. 209–222. https://doi.org/10.1145/1755913.1755935

[25] Alexa VanHattum, Daniel Schwartz-Narbonne, Nathan Chong, and Adrian Sampson. 2022. Verifying Dynamic Trait Objects in Rust. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. 321–330. https://doi.org/10.1145/3510457.3513039

[26] Amit Vasudevan, Sagar Chaki, Limin Jia, Jonathan McCune, James Newsome, and Anupam Datta. 2013. Design, Implementation and Verification of an Extensible and Modular Hypervisor Framework. In *Proceedings of the 2013 IEEE Symposium on Security and Privacy*. 430–444. https://doi.org/10.1109/SP.2013.36